Experiment No-7

Effect of Dropout layers in Preventing Overfitting:-

A bike-rental company wants to predict hourly bike demand across different city locations. The demand depends on temperature, humidity, wind speed, season, holiday status, weekday, hours, and weather conditions. The company wants accurate demand forecasts so that bikes can be redistributed before peak hours.

During initial testing, the company finds that the neural network performs very well on training data but gives poor results on unseen data. This indicates overfitting. To improve generalization, dropout layers will be added to this model. The experiment compares a baseline model without dropout and another model with dropout to observe how dropout reduces overfitting.

Tasks:-

1. Prepare a small bike-demand dataset manually or obtain a suitable bike-sharing dataset from an online source, load it using python, display sample records, and identify demand-related input features and the target count.

2. Preprocess the dataset by encoding categorical variables, removing unnecessary fields, scaling numerical features, and splitting the data into training, validation, and testing sets.

3. Create and train a baseline neural network model without dropout layers, then record training loss, validation loss, and prediction performance.

4. Create a Second neural network model by inserting dropout layers after Selected hidden layers, train it using the Same data and parameters, and compare the learning curves with the baseline model.

5. Test the better model using a sample demand record, display the predicted bike demand, and explain how dropout improves generalization in demand forecasting.

Aim:-

To Study the effect of Dropout layers in preventing overfitting in a neural network for hourly bike-demand prediction, and to compare the performance of a baseline neural network without dropout with a neural network using dropout.

Input Features:-

The following features are used to predict bike demand.

Feature	Description	Type
temp	Temperature	Numerical
humidity	Humidity percentage	Numerical
windspeed	Wind Speed	Numerical
Season	Season of the year	Categorical
holiday	Whether the day is a holiday	categorical / Binary
Weekday	Day of the week	Categorical
hour	Hour of the day	Numerical
Weather	Weather condition	Categorical

Target Variable:

* Count - Total no. of bikes rented during that hour.

Dataset Description:-

The Bike Sharing Dataset contains information about the number of bikes rented at different hours of the day.

* Temperature (temp) - Temperature of the day.
* Humidity (humidity) - Amount of moisture in the air.
* Wind Speed (windspeed) - Speed of the wind.
* Season (season) - Season such as Spring, Summer, Fall, or Winter.
* Holiday (holiday) - Indicates whether the day is holiday.
* Weekday (weekday) - Day of the week.
* Hour (Hour) - Hour of the day.
* Weather (weather) - Weather condition
* Count (count) - Total number of bikes rented.

Target:- Count (bike demand)

The dataset is used to predict bike demand based on weather and time-related conditions.

Algorithm:-

1. Start
2. Load the bike-Sharing dataset using Python.
3. Select input features: temperature, humidity, wind speed, season, holiday, weekday, hour, and weather.
4. Select bike rental count as the target value.
5. Remove unnecessary columns and handle missing values.
6. Encode categorical features into numerical values.
7. Scale the numerical features.

8. Split the dataset into training, Validation, and testing Sets.
9. Build a baseline neural network without dropout.
10. Train the model and record the training and Validation loss.
11. Build another neural network by adding Dropout layers.
12. Train the dropout model using the same data and parameters
13. Compare the training and Validation losses of both models
14. Test the better model on a Sample input and predict bike demand.
15. Observe whether dropout reduces overfitting and improves generalization.
16. Stop.

Program:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from Sklearn.model_selection import train_test_split
from Sklearn.preprocessing import StandardScaler
from Sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

data = pd.read_csv("bike_sharing.csv")

features = ['temp', 'humidity', 'windspeed', 'season', 'holiday', 'weekday',
            'hour', 'weather']

X = data[features]
y = data['count']
X = pd.get_dummies(X, columns = ['season', 'holiday', 'weekday', 'weather'])
```

```
X = X.astype(float)

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.50,
                                   random_state=42)

Scaler = StandardScaler()

X_train = Scaler.fit_transform(X_train)
X_Val = Scaler.transform(X_Val)
X_test = Scaler.transform(X_test)

model1 = Sequential([Dense(64, activation='relu', input_shape=(X_train.
         shape[1],)), Dense(32, activation='relu'),
          Dense(1)
        ])
model1.compile(
        optimizer='adam', loss='mse', metrics=['mae'])

history1 = model1.fit(X_train, y_train, validation_data=(X_Val, y_Val),
                      epochs=30, batch_size=32, Verbose=1)

model2 = Sequential([Dense(64, activation='relu', input_shape=(X_train.shape
         [1],)), Dropout(0.3), Dense(32, activation='relu'),
          Dropout(0.2), Dense(1)])

model2.compile[optimizer='adam', loss='mse', metrics=['mae']]

history2 = model2.fit(X_train, y_train, validation_data=
                      (X_Val, y_Val), epochs=30, batch_size=32,
                      Verbose=1)

loss1, mae1 = model1.evaluate(X_test, y_test, Verbose=0)
loss2, mae2 = model2.evaluate(X_test, y_test, Verbose=0)
print("\nBaseline Model:")
print("Test Loss:", loss1)
print("Test MAE:", mae1)
```

```
print("\nDropout model:")
print("Test Loss:", loss2)
print("Test MAE:", mae2)
plt.plot(history1.history['loss'], label = 'Without Dropout - Training')
plt.plot(history1.history['val_loss'], label = 'Without Dropout - Validation')
plt.plot(history2.history['loss'], label = 'With Dropout - Training')
plt.plot(history2.history['val_loss'], label = 'With Dropout - Validation')
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.show()

Sample = X_test[0].reshape(1, -1)
Prediction = model2.predict(Sample)
print("\nPredicted Bike Demand:", Prediction[0][0])
print("Actual Bike Demand:", Y_test.iloc[0])
```

Result:-

The neural network was successfully trained to predict bike demand. The model with Dropout reduced overfitting and improved prediction on unseen data